

Markov Game (MG) is a mathematical formalism for problems where multiple agents make decisions sequentially to maximize an objective function.

A particular deterministic transition MG instance is defined by the tuple

$$(\mathcal{N}, \mathcal{S}, \mathcal{A}, f, r, D, \gamma, b_0)$$

the set of players
($i \in \mathcal{N}$) playing
the game.

a set of
possible states

the cartesian product
of $\mathcal{A}_i$, where $\mathcal{A}_i$
is a set of player i 's
actions

$r^i(s, a)$ is the scalar
reward function for player
 i , given state s and joint
action a .

the deterministic state
transition function where
 $s' = f(s, a)$ denotes
the transition from state
 s to state s' via action
 a

$D \in \mathbb{N}$ is the number of time steps in the horizon.

b_0 is the initial distribution over states.

$\gamma \in [0, 1]$ is the discount factor

- A policy for player i , π^i , is a mapping from state s , to a distribution over actions $\pi^i: \mathcal{S} \rightarrow \Delta(\mathcal{A}_i)$
- The space of possible policies for agent i is denoted Π^i .
- A joint policy, $\pi = (\pi^1, \pi^2, \dots, \pi^{|\mathcal{N}|})$, is a collection of individual policies for each player.
- The superscript $-i$ is used to mean "all other players".

For example, π^{-i} denotes the policies for all players except i .

- Player i 's objective is to choose a policy to maximize his or her utility or value (V),

$$V^i(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r^i(s_t, a_t) \mid s_0 \sim b_0 \right]$$

Since this objective depends on joint policy, and the reward functions for individual players may not align, with each other, **Markov Games** are not optimization problems where locally or globally optimal solutions are always well-defined. Instead of optima, there are a variety of possible solution concepts.

- The most common solution concept is the **Nash Equilibrium**. A joint policy is a **Nash Equilibrium** if every player is playing a best response to all others.

Mathematically, a best response to π^{-i} is a π^i that satisfies

$$V^i(\pi^i, \pi^{-i}) \geq V^i(\pi^{i'}, \pi^{-i})$$

for all policies $\pi^{i'}$. In a **Nash equilibrium**, this equation is satisfied for all players.

- A relaxation of the **Nash equilibrium** is the **ϵ -Nash Equilibrium**, in which every player has an incentive to deviate that is at most ϵ :

$$V^i(\pi^i, \pi^{-i}) \geq V^i(\pi^{i'}, \pi^{-i}) - \epsilon$$

- A player i 's best response to all other players' policies π^{-i} is a policy that greedily maximizes player i 's utility, holding π^{-i} constant:

$$BR^i(\pi^{-i}) \in \operatorname{argmax}_{\pi^i} V^i(\pi^i, \pi^{-i})$$

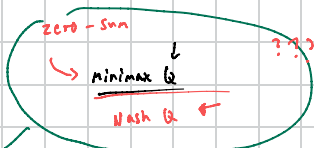
- A useful scalar metric for a distance to Nash is then the sum of all deviation incentives, referred to as the Nash gap or NASH CONV:

$$NASH CONV(\pi) = \sum_{i \in \mathcal{N}} [V^i(BR^i(\pi^{-i}), \pi^{-i}) - V^i(\pi)].$$

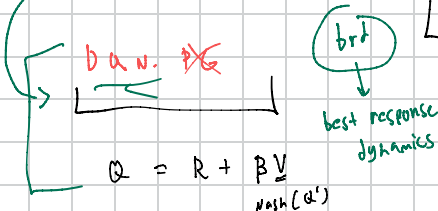
NASH CONV(π) is a nonnegative quantity that, when zero, indicates that π is a joint policy that constitutes a Nash equilibrium by virtue of none of the players having any incentive to unilaterally deviate.

- One particularly important taxonomic feature for MBS is the relationship between the agents' reward functions.
 - In Cooperative games, all agents have the same reward function, $r^i(s, a) = r^j(s, a)$ for all i and j .
 - In a two player sum game, the reward functions of the two players are directly opposed and add to zero, $r^2(s, a) = -r^1(s, a)$.
 - When there are no restrictions on the reward function, the term general-sum is used to contrast with Cooperative, Zero-sum, or other special classes. This work focuses specifically on the two-player Zero-sum case.

Markov game solver



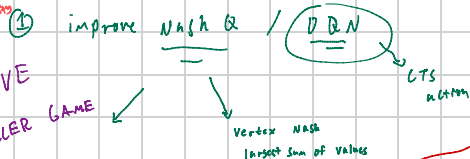
Q-Learning
 Q-iteration
 DDN, PB
 MAX



BIG $\rightarrow$ A1 des $\rightarrow$ A5 debate

low probability

SOLVE SOCCER GAME



BIG $\rightarrow$ A1 $\rightarrow$ A5
 (des) (debate)

brd - best response dynamics

②

A1 des game / debate

interest behavior

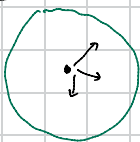
invert LLM ???
 (if familiar)

③

LLM - game

bounded rationality

Example: Soccer Player Game

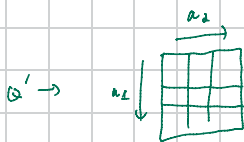


Markov game?

DDN

CPS (structure) action
 action set is structured
 $\downarrow$
 normal form?

$$Q(s, \underline{a_1}, \underline{a_2})$$



???

$$Q = R + \beta \text{Nash}[Q']$$

$\rightarrow$ try on des game

Game with Mixed Strategy Example - Rock Paper Scissors

$$\begin{bmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} + \epsilon$$

How to deal with rounding?

???

how to get mixed Nash Eq. for $Q(s)$?
(value of the game)

cannot introduce, otherwise not indifferent

in different!
Has to be exactly the same!

How to compute value trick?

actions

$$\max(Q \pi_2), \min(\pi_2^T Q), \pi_2^T Q \pi_2$$

definition

Not Nash!

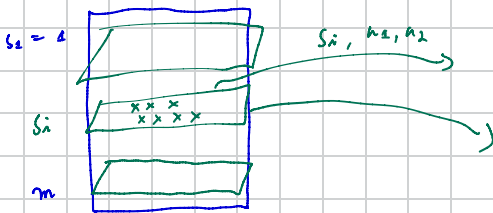
Same!

$$Q = R + \beta \text{Nash}(Q')$$

UPDATE

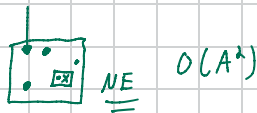
$$Q(s, a_1, a_2) = \text{at most once for every } s'$$

$$Q(s, a)$$



$$\text{Nash}(Q(s'))$$

★ (2) Nash py
scipy
LP



★ (3) not recompute Nash every time.

Solve all NE → freeze → then Q iteration a few times

Syn L

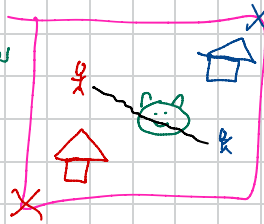
Dog, debate

objective: get to dogs house

People are Sentinels

Dog is agent

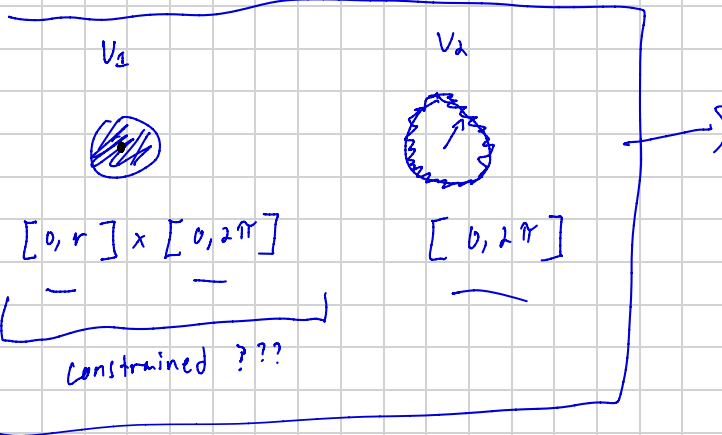
People try to convince dog



$$Dog = w_1 \frac{1}{2} + w_2 \frac{1}{2} + w_0 \cdot \text{dog}$$

DQN $\rightarrow$ $\frac{1}{2} + \frac{1}{2} + \text{bad}$

PG $\rightarrow$???



Markov game

- Solve Soccer game
- Get DQN
- Get DQN to solve this game

My Findings for Week 2

MDP - Markov Decision Process

What is this game?

It's a simplified soccer game on a 7×5 grid. Two players move around, one holds the ball at a time, and the goal is to carry the ball into the opponent's goal (the left or right edge). If you collide with the ball-holder, you steal the ball. The game ends when someone scores or after 100 steps (tie).

What makes it interesting for research?

It's a **Markov game** — a multiplayer generalization of an MDP. Unlike single-player RL where you optimize against a fixed environment, here **both players are learning simultaneously and each player's optimal strategy depends on what the other player does**. This is the core challenge of **Multi-Agent Reinforcement Learning (MARL)**.

The original paper your professor linked (Littman 1994) introduced this exact soccer game as a benchmark for studying these problems.

What's a Nash Equilibrium (NE) here?

A **Nash Equilibrium** is a pair of policies where neither player can improve by unilaterally changing their strategy. In this game:

- Some states have **pure NEs** — one action is clearly best regardless of the opponent
- Some states (especially contested ones near the ball) may have **mixed NEs** — the optimal strategy is to randomize between actions, so the opponent can't exploit you

Your professor specifically wants you to investigate those mixed NE states.

What are we actually building?

Rather than just solving the homework assignment, you're exploring:

1. **Different reward functions** — how does shaping rewards change what the agent learns?
2. **Different algorithms** — REINFORCE, minimax-Q, self-play
3. **Visualization** — making the policies and results visually interpretable
4. **NE analysis** — identifying where mixed strategies emerge and why

Why does the professor want custom reward functions?

The sparse $+1/-1$ reward is theoretically clean but hard to learn from. Designing your own rewards (e.g. small bonuses for possession, progress toward goal, stealing) is a research question in itself — different rewards can produce very different emergent behaviors, some of which may be more interesting than just "win as fast as possible."

Want to start with a specific piece — maybe a visualization of the game environment, or a reward function experiment?

📄 🔊 🌟 🗑️ ↻ 3 minutes ago



Jae-Woo's Key Insight

This is the most practically important message for your work right now:

The assignment (A10) asks for a **deterministic policy** → pure argmax → you naturally get pure MPE (Markov Perfect Equilibrium)

The research asks for a **stochastic policy** → the softmax output is used *as a distribution*, not argmaxed → this is where mixed NE lives

This is why your competition network (which takes the argmax) was fine for the assignment but is the *wrong tool* for the research question. For research you need to:

- Keep the full softmax probability distribution as your policy
- Use algorithms like **minimax-Q** that explicitly solve for mixed strategies
- Evaluate policies by their *expected value* under mixed play, not just win rate

So what should you actually implement?

The group's priority for the next meeting is **minimax-Q** to find mixed MPE. That means:

- At each state, instead of picking the best action against a fixed opponent, **solve a linear program to find the mixed strategy that maximizes your worst-case value**
- This is the Littman 1994 algorithm your professor linked

Want me to build a minimax-Q implementation for the soccer environment, along with visualizations showing where the mixed strategies emerge (i.e. which states have high-entropy policies)?

📄 🗨️ 📌 🗨️ 🔄 2 minutes ago

Second Meeting 9/10

$\pi_1 Q$ & $\pi_2 Q$ may not be the same

Q - Matrix - Continue

Visualize the states where one or both players use mixed strategies

- produce all $\mathbb{Z}^2$, and compare to human interaction

$(0, \pm, \pm, \pm, \pm)$

Would these happen
regardless of where
they are? OR does
it matter with
position?

	0	1
0		
1	X ← X	
	↓	↓

	mixed Nash	
	0	1
0	X →	
1	↓	← X

???

Find
Indifferences,
may be pure
strategy mix

X
.....
X

interesting ???

- Create Original Markov Game Table

- Minimax vs. Always move Left

- Minimax vs. Best Response (before move,

I know what
move you are
going to make,
but I don't
know strategy
and need to
compute reward)

- Minimax vs. Random

- Implement
Stochastic Transition

Exact Solver

→ Planning → minimize value iteration
→ Learning → Minimax Q

Grid
Spaces

→ { mixed stage games - tabular
interesting policy

Homework: Simple Exercise

AI - dog, B I G

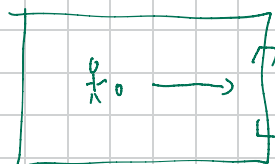
Battling Influencer Game

- If there are two or more influencers, we want to convince the audience to be as close to the target opinion, others are exaggerating to resist this.



↑
general sum game

convex compact



↑
zero sum game

If houses still can do this

w/ different x & y coordinate,
this can converge

DQN



★ Solve the stage game?

PG

Protonin??

RGJNEOR
AdL
p p O

Where do we need to solve
the stage game to get this
into the solution of the Markov game?

can this converge on
the dog game??

① DQN: $Q = R + P(NE)(Q')$

$\approx NN(W)$

minimize by choosing
weights

$\frac{1}{2} (Q - \hat{Q})^2$

↑
we don't have Q

↑
difference small as possible

want to be exactly 0

$W = W - \alpha \frac{\partial L}{\partial W}$

minimize weights $\frac{1}{2} ((\hat{R} + B NE(Q')) - \hat{Q})^2$

overfit

→ Deep Q Learning

① Freeze target, then try to get close to target
(target & freeze Q')

② Experience replay minibatch gradient descent

FOCUS

* If you run into any numerical issues, record them

$$w = w - \alpha \frac{\partial L}{\partial w} \quad \begin{array}{l} (x, y) \text{ supervised} \\ \downarrow \\ (\underbrace{s}_{\text{label}}, \underbrace{r + \beta V(w)}_{\text{feature}}) \text{ RL [Reinforcement Learning]} \end{array}$$

Inner loop: Q
Outer loop: $\bar{Q}$

Train in PyTorch!

PG (A2C)

$$\begin{aligned} \max_w V(w) & \quad \text{policy network } \pi(w) \\ \frac{\partial V}{\partial w} &= E \left[\frac{\partial}{\partial w} Q \log \pi(w) \right] \\ &= E \left[\frac{\frac{\partial \pi}{\partial w}}{\pi(w)} Q \right] \\ &= \int \frac{\partial \pi}{\partial w} Q \pi(w) d\pi \end{aligned}$$

Loss function: $Q \log(\pi(w))$

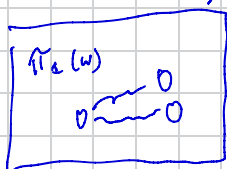
now do Cross-Entropy Loss

$$w = w - \alpha \frac{\partial \mathcal{L}}{\partial w} \rightarrow$$

(x, y)

$(s, R_1 + \beta R_2 + \beta^2 R_3 + \dots)$

Need randomness to generate R



$(s', R_2 + \beta R_3 + \beta^2 R_4 + \dots)$

$(s'', R_3 + \beta R_4 + \beta^2 R_5 + \dots)$

$\vdots$

minibatch
(optimization
tricks)

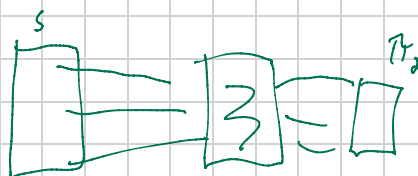
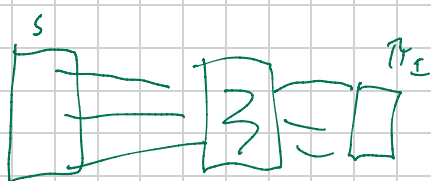
T

(s^{T-1})

(s^T)

When you have two players, how do you organize the network?

(1)



(2)



Only applies to policy
gradient algorithm

⑤



freeze weights to 0

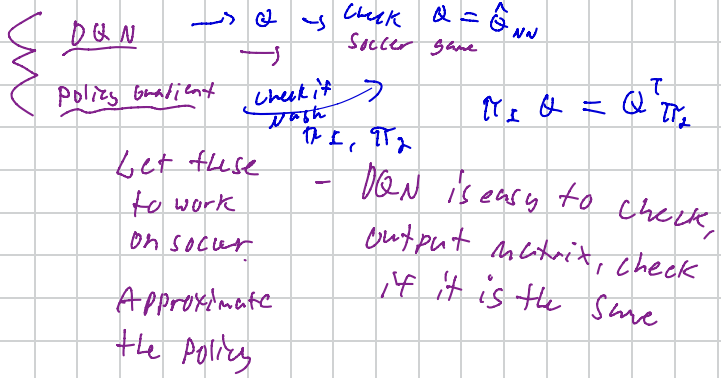
Separate subnetworks to control each player.

- In each training, are there any differences.
- How would these types of training play against each other?

DO NOT HARDCODE THIS?

The key algorithm must be experimented by you! Adjust the learning rate. Use packages for network training! Use LLM to come up with algorithm. Use packages just for training. Implement D&V manually. Separate hard code for training function and reward function!

Next week →



Next: Start to apply on AI - dog game
